From GitHub change to a governed workspace

Read top to bottom. Solid arrows: workflow or API operations. Dashed purple arrows: authentication across trust boundaries.

GitHub repository + GitHub-hosted Actions

AUTHOR & REVIEW

1 GitHub user
Platform admin or requesting data team

2 Edit declared configuration
Admins: platform/environments + access + compute
Teams: workloads/<team>/dev-demo.yaml
Reusable Terraform: environments/ + modules/

3 Pull request, review and merge
Relevant PRs and main pushes trigger validation

Automatic: offline validation only
Python schema / semantic checks + unit tests
Terraform fmt / validate + mocked provider tests
No live plan, apply, destroy or cloud credentials

4 Actions > Provision environment YAML
Choose main, environment=dev, workspace=demo
deploy=false: plan only; deploy=true: gated apply
Fresh downstream plans require applied upstream state

Merge does not deploy

MANUAL EXECUTION ON MAIN / dev-demo

5 Verify policy, then plan / apply
Manual main run; initiating + rerun actors are admins
deploy=true + apply opt-in + confirm dev/demo
Each stage applies its exact same-run saved plan
Python renders isolated inputs; Terraform executes

6 01-infra
AWS network, IAM, S3 and monitoring

7 02-account
Register AWS config; create workspace + groups

8 02-workspace-access
Apply persona feature entitlements

9 04-workspace
Create policies, analytics SQL and compute ACLs

Plan artifacts + serialized execution
Plan + manifest retained 1 day; compute plan expires in 24h
Source / input / backend / state bindings checked
Target concurrency prevents overlapping lifecycle runs

OIDC

Customer AWS account

AWS IAM / STS
OIDC token -> AssumeRoleWithWebIdentity

AWS APIs + Terraform backend
Provision resources; read / write S3 state
Separate state key per environment/workspace/root

Databricks-managed control plane

Account API / OAuth M2M
Client ID + GitHub environment client-secret
Account credentials, network, storage, groups

Workspace API / OAuth M2M
workspace_url + workspace_id from account state
Feature entitlements, cluster policies, SQL ACLs
Account and workspace providers are separate

Runtime compute resides in the customer AWS VPC.
See page 2 for the network and data boundaries.

Separate manual data-permissions flow
Existing objects -> root05 plan -> reviewer-gated apply

AWS network and Databricks service boundaries

dev/demo YAML: eu-west-1, VPC 10.40.0.0/16, two availability zones, single NAT. Runtime paths are architectural inferences.

Databricks control plane

Account services
AWS configuration registration
Workspace lifecycle + account groups
Cross-account provisioning role

Workspace services
UI / API, policies and SQL definitions
Jobs module exists; demo runner
does not create jobs or clusters

Outbound control connectivity via NAT

Unity Catalog governance
Separate root03 integration path
Metastore assignment; credentials
External locations; optional objects
Root05: grants on existing objects

Logical SaaS boundary; these services are outside your customer VPC.

Customer AWS account / eu-west-1

AWS IAM roles
Deployment role (GitHub) != Databricks cross-account role
UC storage roles are owned separately by root03

S3 Terraform state backend (prerequisite)
databricks/<environment>/<workspace>/<root>.tfstate
Six isolated states for dev/demo; backend outside VPC

Customer-managed VPC / 10.40.0.0/16

Availability zone eu-west-1a

Private workspace subnet A
10.40.0.0/20 | workspace security group
Classic / SQL compute placement is service-managed

Endpoint subnet A: 10.40.34.0/24

AWS API traffic

Availability zone eu-west-1b

Private workspace subnet B
10.40.16.0/20 | workspace security group
Classic / SQL compute placement is service-managed

Endpoint subnet B: 10.40.35.0/24

Public NAT subnet A
10.40.32.0/24 | single NAT gateway
Both workspace subnets share egress
Internet gateway -> public services

AWS service endpoints
S3 gateway endpoint attached to workspace route tables
STS + Kinesis interface endpoints across both AZs
Private DNS; endpoint SG permits HTTPS from workspace SG

S3 workspace + data storage
Root storage, landing and UC managed-data buckets
Buckets are AWS services outside VPC subnets
UC credentials / IAM roles authorize data access
S3 gateway endpoint supplies the private route

Monitoring + optional services
CloudWatch dashboard and NAT alarms: enabled
Customer-managed KMS: disabled in demo YAML
Kinesis stream: optional; endpoint exists independently
Budget email alerts only when recipients are configured

Boundary notes
No runnable Databricks PrivateLink module.
AWS endpoints do not make the UI private.
Single NAT is not AZ-resilient egress.
Cluster policies alone launch no clusters.
SQL warehouse apply may start compute.

State ownership, access layers and lifecycle

Each layer owns a different concern. Compute access does not grant data access; group membership remains external.

S3 Terraform ownership boundaries / S3 prefix: databricks/dev/demo/

STATE / EXECUTION ROOT	OWNS	INPUT / DEPENDENCY	AUTOMATION SCOPE
01-infra.tfstate	VPC, subnets, NAT, SGs, endpoints; IAM, S3, optional KMS / stream, metrics	Environment YAML + supplemental inputs	Manual provision: dev / staging
02-account.tfstate	Workspace creation + AWS registration; account groups + workspace assignments	01-infra outputs; identity checked	Manual provision: dev / staging
02-workspace-access.tfstate	Bootstrap persona entitlements only Workspace / SQL / compute feature flags	02-account IDs + live identity checks	dev/demo after account apply
03-data-access.tfstate	Metastore assignment, UC credentials, UC IAM roles, locations, optional objects	01 + 02; IAM trust uses generated IDs	Separate Terraform root
04-workspace.tfstate	Demo: policies, SQL warehouse, ACLs Other callers: optional jobs / profiles etc.	02 + applied entitlements for demo	dev/demo after feature apply
05-permissions.tfstate	Grants on EXISTING catalog/schema/table; never duplicate bootstrap group ownership	Verified identities, target + existing objects	Separate manual plan / gated apply

Root04 demo bypasses root03 only without UC log/init-script volumes. Legacy root04 normally reads root03. Shared UC scopes need one grant owner.

Persona access / dev-demo-* account groups

PERSONA	WORKSPACE / SQL	CLUSTER POLICY	ANALYTICS SQL
admins	ADMIN; UI + SQL	Workspace-admin policy access	CAN_MANAGE
engineers	USER; UI + SQL	engineering-standard: CAN_USE	CAN_USE
analysts	USER; SQL only	None	CAN_USE
ml	USER; UI + SQL	ml-standard: CAN_USE	CAN_USE

Engineers / analysts / ML: unrestricted cluster and pool creation disabled by entitlements.

Approval and lifecycle boundaries

Provision / retire dev-demo: single-admin exception

- Manual main + verified actors + explicit operation opt-in
- Exact target confirmation + same-run saved plan
- Root05: required reviewers; no self-review or admin bypass
- Separate permissions apply opt-in remains mandatory
- README reports root05 protection absent; live settings unverified

Retirement is a separate, explicitly confirmed operation

Retire root03 / root05 data ownership separately before account / infrastructure teardown.
Demo workflow order: compute (04) -> feature entitlements -> account (02) -> infrastructure (01).
Retain S3 and KMS by default. Optional storage purge requires its own confirmation and empty-state checks.
Preserve backend history and KMS; preview-only demo stops after compute / feature previews.

Implementation scope

dev/demo: infrastructure + identities + features + persona compute
Other dev / staging: environment runner provisions roots 01 / 02
Production: templates / scaffolds; Azure: placeholder
Source index and interpretation notes accompany the editable SVGs.